Ders 9 Genişletilmiş Notu: Yazılım Süreç İyileştirme: CMMI, People CMM ve Test Güdümlü Geliştirme

Ders 9 Genişletilmiş Notu: Yazılım Süreç İyileştirme: CMMI, People CMM ve Test Güdümlü Geliştirme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Hedefi: Kalite İyileştirme Nedir?
- 2. Standartların Sınıflandırılması
- 3. Kaliteyi Sağlamak İçin Yapılanlar ve Sertifikasyon
- 4. Yazılım Geliştirme Ortamı Sertifika Sistemleri
- 5. CMMI: Tanım, Olgunluk ve Süreç İyileştirme Döngüsü
- 6. Firmalar Neden Sertifika İster? Akreditasyon Benzetmesi
- 7. CMMI Olgunluk Düzeyleri
- 8. Türkiye'den CMMI Örnekleri
- 9. CMMI Anahtar Süreç Alanları
- 10. CMMI'de Süreç Alanı Grupları
- 11. People CMM
- 12. CMMI Bölümünün Toparlanması
- 13. Çevik Yaklaşımlar ve XP
- 14. Çevik Modellerde Kalite: Geleneksel Modellerden Farklar
- 15. Statik ve Dinamik Doğrulama
- 16. Test Güdümlü Yazılım Geliştirme (TDD)
- 17. TDD Üzerine Bir Araştırma

1. Dersin Hedefi: Kalite İyileştirme Nedir?

Bu ders vize sınavından sonraki ilk dersti. Hoca, sınava dair kısa değerlendirmenin ardından konuya geçti. Ders iki ana parçadan oluşuyordu: önce yazılım evlerinde (software house) süreç iyileştirme ve sertifikasyon, sonra çevik (agile) modellerde kalite işlerinin nasıl yürüdüğü.

Hoca bugünkü dersin hedeflerini şöyle sıraladı:

- Yazılım süreç iyileştirme (software process improvement) dediğimizde ne anladığımızı görmek.
- Yazılım evleri için hangi sertifikasyonların bulunduğunu anlamak.
- Yazılım geliştirme sürecinde hangi anahtar süreç alanlarının (key process areas) olduğunu ve bunların düzeylerle nasıl ilişkilendirildiğini görmek.
- Bugün sertifikasyonun içine alınmış olan, çalışanlara yönelik yetenek modelini (People CMM) incelemek.

2. Standartların Sınıflandırılması

Hoca önce önceki derslerden bir hatırlatma yaptı. Yazılım kalitesini sağlamak için yapılan işlerin başında standart koymak ve bu standartlara uyarak yazılım geliştirmek gelir. Standartlar iki şekilde sınıflandırılmıştı:

- **Kaynağına göre:** Standardın menşei nedir, hangi kurum veya organizasyon onu oluşturmuş ve desteklemektedir? IEEE'nin, ISO'nun ya da ABD Savunma Bakanlığı'nın (DoD) oluşturduğu standartlar gibi.
- **Yazılım geliştirme sürecine göre:** Standart hangi süreç adımını desteklemek için konmuştur? Yazılım geliştirme planını destekleyen, yazılım gereksinimini kontrol altında tutan, yazılım tasarım belgelerini düzenleyen standartlar gibi.

IEEE standartlarında bu iki sınıflandırma birlikte görülür. Hoca örnek olarak öğrencilerin projede kullandığı IEEE 830 standardını verdi. Gereksinim belgesini hazırlarken bu standart kullanılmıştı. Hoca standardı birebir ya da özetleyerek kullanabileceklerini, genellikle özetleyerek kullanmalarını istediğini hatırlattı. IEEE 830'un kaynağı IEEE'dir. Yazılım geliştirme sürecine göre tipi ise yazılım gereksinim analizini kontrol altında tutan bir standarttır.

3. Kaliteyi Sağlamak İçin Yapılanlar ve Sertifikasyon

Hoca, kaliteyi sağlamak için şimdiye kadar konuşulan aktiviteleri süreç sırasıyla özetledi:

- **Geliştirmeye başlamadan önce:** Standartlar ve bir yazılım geliştirme süreç modeli belirlenir.
- **Geliştirirken:** Gözden geçirme (review), inceleme (inspection) ve test yapılır.
- **Tamamlarken ve tamamladıktan sonra:** Konfigürasyon yönetimi yapılır.
- **Sonraki süreçlerde:** Ölçümler yapılarak yazılım evinde bir ölçüm arşivi oluşturulur.

Bunlara ek olarak, yazılım evinin belirli koşulları sağladığını ve kendini ispatladığını göstermesi için bir sertifika alması uygun olur. Hoca burada önemli bir noktaya dikkat çekti: Bir kurum sertifikasyon kurumuna doğrudan başvurmasa bile, sertifikasyonun koşul listesine bakarak kendini daha kaliteli çalışan bir yazılım evi, bir yazılım ekibi hâline getirebilir.

4. Yazılım Geliştirme Ortamı Sertifika Sistemleri

Hoca yazılım geliştirme ortamı için üç sertifika sisteminden söz etti:

- **CMM ve CMMI:** ABD menşelidir. CMMI, CMM'nin ilerlemiş hâlidir. CMM, yetenek olgunluk modeli (Capability Maturity Model) anlamına gelir.
- **SPICE:** Avrupa'da yazılım sürecinin iyileştirilmesi için gerçekleştirilecek kontrol adımlarını tanımlar. Hocaya göre CMMI ne yapıyorsa SPICE da onu yapar. Farkları gerçekleştirme biçimleri ve hitap ettikleri yerlerdir.
- **ISO:** Bu işin nasıl olması gerektiğine dair, proje yönetim süreci ve gerçekleştirme süreci ile ilgili tanımları ISO 9000 ailesinin bölümlerinde yapmıştır.

  Ders sonrası not: Altyazıda açılım "capability majority model" olarak geçiyor. Doğrusu Capability Maturity Model'dir; CMMI'deki "I" ise Integration (bütünleşme) anlamına gelir. Hocanın andığı "ISO 9003" büyük olasılıkla ISO 9001'in yazılıma uygulanması için kılavuz olan ISO 9000-3'tür; bunun bugünkü karşılığı büyük olasılıkla ISO/IEC 90003'tür.

  Ek bilgi (derste anlatılmadı): SPICE (Software Process Improvement and Capability dEtermination) büyük olasılıkla ISO/IEC 15504 standardı olarak yayımlanmıştır. Bu standardın yerini bugün büyük olasılıkla ISO/IEC 33000 serisi almıştır. CMMI'den farklı olarak SPICE süreçleri tek tek yetenek düzeyiyle değerlendirmeye ağırlık verir.

5. CMMI: Tanım, Olgunluk ve Süreç İyileştirme Döngüsü

5.1 Süreç Odaklı Kalite

CMMI'nin temel varsayımı şudur: Yazılım sektöründe ürünün kalitesini ağırlıkla onu üreten sürecin kalitesi belirler. Bu yüzden CMMI süreç odaklı bir kalite yaklaşımıdır. Hoca iki tanım verdi:

- CMMI, etkin bir yazılım sürecinin anahtar elemanlarını tanımlayan bir çerçeve modeldir.
- CMMI, süreç olgunluğunu değerlendirmek için bir ölçüm aracıdır.

5.2 Olgunluk: Yeni Kurulmuş Bir Ekip Sertifika Alamaz

Hoca "olgunluk" kelimesinin anlamını bir örnekle açtı. Bugün hizmete başlamış bir yazılım ekibi CMMI sertifikası alamaz. Bu ekip "gereksinim analizini de proje yönetimini de belli koşullara uyarak yapıyoruz" dese bile, belli bir proje arşivinin olması gerekir. Yani sertifika için hem yetkinlik hem de olgunluk gerekir.

> **Hocanın vurgusu:** CMMI için yalnız doğru işleri yapmak yetmez; bunların zaman içinde birikmiş bir proje geçmişiyle kanıtlanması gerekir. Hem yetkinlik hem olgunluk aranır.

5.3 Süreç İyileştirme Döngüsü

Süreç iyileştirme bir döngü hâlinde ilerler:

- Yazılım geliştirme sürecinde ölçümler yapılır.
- Ölçüm sonuçları analiz edilir.
- Süreçte bir değişiklik yapılır.
- Döngü yeniden ölçümle devam eder.

5.4 Düzeyler Silsilesi ve CMMI'nin Kökeni

CMMI bir düzeyler silsilesidir. Sertifika almaya niyetlenen bir yazılım evi ya da bir kurumun yazılım ekibi önce CMMI'deki koşulları öğrenir, gerçekleştirmeye çalışır ve basamakları sırayla çıkar. Hoca bunu şöyle anlattı: Bir kurum birdenbire dördüncü düzeyi, sayısal yönetilebilirliği hedeflemez. Önce bir başlangıç, niyet aşaması vardır. Sonra yönetilebilir olacak çalışmalar yapılır. Daha sonra tanımlı (defined), sayısal yönetilen (quantitatively managed) ve en iyileştiren (optimizing) düzeylere geçilebilir.

CMMI, ABD'de Carnegie Mellon Üniversitesi bünyesindeki Software Engineering Institute (SEI) merkezli olarak başlatılmış ve oradan yönetilmektedir. Bazı ülkelerde temsilciler aracılığıyla başvuru yapılır. Başvuru sonucunda kurum denetlenir. Denetimde kurumun yalnız birinci düzeyin koşullarını mı, yoksa ikinci, üçüncü ya da dördüncü düzeyin koşullarını mı sağladığına karar verilir ve sertifika buna göre verilir.

> **Ders sonrası not:** CMMI büyük olasılıkla 2013'ten beri SEI yerine CMMI Institute tarafından yönetilmektedir; CMMI Institute de büyük olasılıkla 2016'da ISACA bünyesine katılmıştır. Hocanın anlattığı SEI merkezli yapı modelin kökenini doğru yansıtır.

6. Firmalar Neden Sertifika İster? Akreditasyon Benzetmesi

Hoca firmaları sertifikaya iten güçleri şöyle açıkladı:

- Bazı ihalelere girebilmek için firmanın belli bir düzeyde olduğunu gösteren sertifika istenir.
- Geliştirilen yazılımın güvenirliğini göstermek için sertifikaya ihtiyaç duyulur.
- Örneğin askeri ihalelere girebilmek için firmaların CMMI'de üçüncü düzeyde olması gerekir.
- Bazı ticari işlerde de belli bir yeterlilik düzeyi koşulu aranır.

Hoca bunu her yıl derslerde üniversite akreditasyonuna benzettiğini söyledi. Bölümlerin kendilerini ispatlaması ve mezunlarının dışarıda kabul görmesi için bir akreditasyon sürecinden geçmeleri gerekir. Türkiye içinde bunun karşılığı MÜDEK'tir. Birçok üniversitenin sayfasında MÜDEK akreditasyonunun hangi yıl alındığı ve hangi yıla kadar sürdüğü yazar. Uluslararası alanda ise ABD menşeli ABET akreditasyonu vardır. Mühendislik okulları ABET'e başvurur, oradan gelen denetçilerin denetiminden geçer ve akreditasyonu alır.

Yazılım evleri için de durum benzerdir. Bir yazılım evinin sayfasında "çok iyi bir yazılım eviyiz, yazılımlarımız şu kadar yerde kullanılıyor, paydaşlarımız ve kullanıcılarımız bunlardır" demesi bir şeydir. "Bu kurum CMMI şu düzeydedir" demek ise farklıdır, çünkü ikincisi ölçülebilir bir değerlendirmeye dayanır: Bir ölçüm yapılır, neyin ne kadar sağlandığı analiz edilir, kurum bir değişim ve dönüşüm geçirerek iyileşir ve yeniden ölçülür.

> **Hocanın vurgusu:** Sertifika kalıcı değildir. Örneğin dördüncü düzeydeki bir firma üç yıl sonra değişim geçirdiğinde yeniden ölçülür ve aynı düzeyi tutturup tutturamadığına bakılır. Düzeyi korumak süreklilik ister.

7. CMMI Olgunluk Düzeyleri

7.1 Düzey 1: Başlangıç (Initial)

Başlangıç düzeyinde firma yalnızca bir niyet göstermiştir. Bu düzeyin özellikleri şunlardır:

- Yazılım süreci gelişigüzel ve kaotiktir.
- İşler kişisel gayretlere bağlı olarak yürür.
- Kriz durumunda var olan planlar terk edilir.
- Yazılım süreç yetenekleri önceden kestirilemez.

7.2 Düzey 2: Yönetilen (Managed)

İkinci düzeyde proje yönetimi kontrolleri kurulmuştur ve proje yönetimi ayrıntılı olarak yapılır. Projeler belgelenmiş planlara göre yürütülür: Bir proje başlarken belgesiyle birlikte bir plan oluşturulur, proje bu plana uygun yönetilir ve plan takibi yapılır. Gereksinimler belirlenir ve yönetilir. Gereksinimlerle ilişkili ürünler oluşturulur ve denetlenir. Proje yönetim sistemi proje sürecini etkin bir biçimde kontrol eder. Ölçümler yapılır. Ürünler ve hizmetler, kendilerine özgü gereksinim, standart ve hedefleri sağlayacak şekilde gerçekleştirilir.

> **Hocanın vurgusu:** İkinci düzeyde konfigürasyon yönetimi mutlaka yapılmalıdır.

7.3 Düzey 3: Tanımlı (Defined)

Düzeyler basamak basamak gittiği için üçüncü düzeydeki bir kurum ikinci düzeyin koşullarını zaten sağlıyor demektir. Yani bu kurumda proje yönetimi, gereksinim yönetimi ve konfigürasyon yönetimi vardır. Üçüncü düzeyin özellikleri şunlardır:

- Süreçler daha iyi tanımlanmış ve daha iyi anlaşılmıştır.
- İşlemler kurum bazında tutarlılıkla gerçekleştirilir.
- Tüm projelerde, kurum bazındaki standart yazılım geliştirme süreci projeye uyarlanmış olarak kullanılır.
- Risk yönetimi yapılır.

Hoca risk yönetiminin anlamını şöyle açtı: Şu anda bir problem yoktur, ama ileride ortaya çıkabilecek problemlerin kaynakları nelerdir? Kurum bunları araştırıp bütün projelerinde takip edebiliyor mu? Ancak bu durumda tanımlı düzeyden söz edilebilir.

> **Hocanın vurgusu:** Risk yönetimini slayta yazmadığını, ama tanımlı düzey için bunun önemli olduğunu belirtti.

7.4 Düzey 4: Sayısal Yönetilen (Quantitatively Managed)

Dördüncü düzeyde sürecin başarımı için önemli alt süreçler seçilir. Bu alt süreçler istatistiksel ve diğer nicel ölçüm teknikleri kullanılarak kontrol edilir. Kalite ve süreç performansı için sayısal hedefler oluşturulur ve bu hedefler süreçlerin yönetiminde bir ölçüt olarak kullanılır.

Hoca bu düzeyi önceki derslerde görülen yazılım ölçütleri (software metrics) konusuna bağladı. Dördüncü düzey, bu ölçümleri yapmış olmayı, bir arşiv oluşturmayı ve bu ölçümleri daha sonraki yazılım geliştirmede kullanabilmeyi gerektirir. “Bir yazılım geliştirdik, kaç satır olduğunu ölçtük” demek yeterli değildir. Asıl soru şudur: Bu ölçümü başka projelerde kullanabiliyor muyuz, bir arşivimiz var mı?

İkinci ve üçüncü düzeylerde de yazılım ölçütlerine bir giriş vardır. Fark şuradadır: Sayısal yönetilen bir kurum yeni bir projeyi yönetirken projenin ne büyüklükte olacağını başta tahmin eder. Sonra eski projeler arşivine dönüp bu büyüklükteki projeleri ne kadar maliyetle gerçekleştirdiğine bakabilir. Ancak bu bilgi elde edilebiliyorsa dördüncü düzeyden söz edilebilir.

> **Hocanın vurgusu:** Dördüncü düzey yalnız ölçüm yapmak değildir. Ölçümler arşivlenmeli ve yeni projelerin tahmininde dayanak olarak kullanılmalıdır.

7.5 Düzey 5: En İyileştiren (Optimizing)

Beşinci düzeyde şirkette ya da yazılım ekibinde sürekli en iyileştirme çabasının olup olmadığına bakılır. Sorulan sorular şunlardır:

- Süreçten gelen sayısal geri beslemelerden ve yeni teknolojilerden faydalanarak sürekli süreç iyileştirmesi yapılıyor mu?
- Sürekli iyileştirme için yenilikler, yenilikçilik var mı?
- Çalışanlar yeni bir fikir ürettiğinde bu fikir nasıl değerlendiriliyor, dikkate alınıyor mu?

Yeni fikir üretebilmek, sürekli süreç iyileştirmesi yapmak ve sürekli değişen süreçler içinde bulunmak beşinci düzeyi gösterir.

7.6 Düzeylerin Özeti

Düzey	Ad	Temel özellik
1	Başlangıç (Initial)	Gelişigüzel, kaotik süreç; kişisel gayret; krizde planlar terk edilir
2	Yönetilen (Managed)	Belgelenmiş planlarla proje yönetimi; gereksinim yönetimi; ölçüm; konfigürasyon yönetimi
3	Tanımlı (Defined)	Kurum bazında standart ve uyarlanmış süreç; tüm projelerde risk yönetimi
4	Sayısal Yönetilen (Quantitatively Managed)	İstatistiksel ve nicel kontrol; ölçüm arşivine dayalı tahmin
5	En İyileştiren (Optimizing)	Sayısal geri besleme ve yenilikçilikle sürekli süreç iyileştirmesi

8. Türkiye’den CMMI Örnekleri

Hoca konuyu somutlaştırmak için Türkiye’deki firmalarda CMMI’nin ne kadar önem taşıdığına değindi. Hata açısından kritik yazılımlarla uğraşan ekipler, özellikle askeri projelerde çalışan kurumlar CMMI 3 sertifikasını almak durumundadır. Hocanın verdiği örnekler şunlardır:

- TÜBİTAK MAM, askeri yazılım üreten firmalara test yaptığı için dördüncü düzey sertifika almıştı.
- Huawei’de de böyle bir sertifikasyon çalışması vardır. Bu, hata toleransı az olan kurumlarda görülür.
- Askeri yazılım üreten firmalar arasında Aselsan anıldı.
- Bankacılık uygulamaları geliştiren Cybersoft dördüncü düzeydeydi.
- Beşinci düzeyde olan firma olarak Milsoft geliyor; Milsoft yıllardır beşinci düzeydedir.

Hoca Milsoft örneği üzerinden beşinci düzeyin bedelini de anlattı. Firmanın projelerinde teslim ve proje tamamlama süreleri diğer firmalara göre biraz daha uzundur. Firma, beşinci düzeyde olma sürekliliğini sağlamak için kaynaklarını kanıt üretmeye ayırmak, bu kanıtlar için belgeler hazırlamak zorundadır. Bir firmayı değerlendirirken, buna rağmen beşinci düzeyde kalabildiğine bakmak gerekir.

> **Hocanın vurgusu:** Bu bilgiler sürekli takip edilmelidir. “Üç yıl önce CMMI 4’teydi” demek yetmez; bugün hangi düzeyde olduğuna ve firmanın buna hâlâ dikkat edip etmediğine bakılmalıdır.
>
> **Ders sonrası not:** Hocanın verdiği firma düzeyleri ders tarihindeki (Aralık 2020) bilgilerdir. Güncel düzeyler büyük olasılıkla CMMI Institute’un yayımladığı değerlendirme sonuçları listesinden kontrol edilebilir.

9. CMMI Anahtar Süreç Alanları

Aradan sonra hoca anahtar süreç alanlarına geçti. Anahtar süreç alanı, CMMI'de her bir düzeye ulaşmak için yazılım geliştirme sürecinde yapılması gereken işleri anlatır. Yani hangi düzeyde hangi işlerin yapılması gerektiğini söyler.

9.1 Düzey 1

Birinci düzey için bir anahtar süreç alanına gerek yoktur. Bu düzeyde her şey zaten bireysel çabalara göre yürür.

9.2 Düzey 2: Yönetilen

Yönetilen düzeyde olmak için şunlar gerekir:

- Gereksinim yönetimi: Gereksinim analizi yapılmalı, gereksinim belirtimi (requirement specification) oluşturulmalı; gereksinim yönetimi bütünüyle sağlanmalıdır.
- Proje planlama.
- Projenin izlenmesi ve kontrolü.
- Tedarikçilerle sözleşme yönetimi.
- Ölçümleme ve analiz.
- Süreç ve ürün kalite güvencesi.
- Konfigürasyon yönetimi.

Hoca ölçümleme konusunda dördüncü düzeyle farkı yeniden vurguladı. İkinci düzeyde de ölçüm yapılır, ama bu ölçüm süreklilik göstermez. Bir sonraki projede eski ölçümlere dayanarak tahmin yapılmaz.

9.3 Düzey 3: Tanımlı

Tanımlı düzeyde şu süreç alanları bulunur:

- Gereksinimleri geliştirme: Gereksinim analizi bütün projeler için sürekli hâle getirilir.
- Teknik çözüm.
- Ürün entegrasyonu: Yazılım ekibinde geliştirilen ürünlerin birbiriyle uyumlu, bütünleşik ve birlikte çalışabilir olmasına dikkat edilir.
- Doğrulama ve onaylama (verification, validation).
- Kurumsal süreç tanımlama ve odaklanma.
- Kurumsal eğitim.
- Entegre ürün ve süreç geliştirme için bütünleşik proje yönetimi.
- Risk yönetimi.
- Bütün ürünlerin geliştirildiği ortak, bütünleşik ve kurumsal bir altyapı.

Hoca üçüncü düzeyi şöyle özetledi: İkinci düzeydeki işlemler (gereksinim analizi, proje yönetimi, doğrulama) kurumsal yapıda, bütün projelerde aynı şekilde takip edilir ve bütünleşik ürünler geliştirilir.

> **Hocanın vurgusu:** Üçüncü düzeydeki bir firmanın en önemli tarafı, risk yönetiminin mutlaka bütün projelerde yapılmasıdır.

9.4 Düzey 4: Sayısal Yönetilen

Bu düzeyin anahtar süreç alanları kurumsal süreç başarımı ve sayısal (niceliksel) proje yönetimidir. Sayısal proje yönetimi, ölçüm yapmak ve bu ölçümleri yeni projelerde dayanak noktası olarak kullanmak demektir. Eski proje arşivlerine bakarak yeni projelerin maliyeti ve yapısı hakkında karar verilir. Hocaya göre dördüncü düzeyin en önemli anahtar süreç alanı sayısal proje yönetimidir.

9.5 Düzey 5: En İyileştiren

Beşinci düzeyin anahtar süreç alanları kurumsal yenilenme ve yaygınlaştırma ile nedensel analiz ve çözümlemedir. Hocaya göre bu düzeyi farklılaştıran en önemli işlem kurumsal yeni-

lenmedir: Kurum yeni teknolojilere, yeni yöntemlere, yeni süreç modellerine ve yeni çözümlere ne kadar yaklaşıyor? Bunun sağlanması beklenir.

9.6 Anahtar Süreç Alanlarının Özeti

Düzey	Anahtar süreç alanları
1 Başlangıç	Yok
2 Yönetilen	Gereksinim yönetimi, proje planlama, proje izleme ve kontrol, tedarikçi sözleşme yönetimi, ölçümleme ve analiz, süreç ve ürün kalite güvencesi, konfigürasyon yönetimi
3 Tanımlı	Gereksinimleri geliştirme, teknik çözüm, ürün entegrasyonu, doğrulama ve onaylama, kurumsal süreç tanımlama ve odaklanma, kurumsal eğitim, bütünleşik proje yönetimi, risk yönetimi, kurumsal entegrasyon altyapısı
4 Sayısal Yönetilen	Kurumsal süreç başarımı, sayısal proje yönetimi
5 En İyileştiren	Kurumsal yenilenme ve yaygınlaştırma, nedensel analiz ve çözümleme

10. CMMI'de Süreç Alanı Grupları

Hoca, eskiden süreç yönetimi (process management), proje yönetimi (project management), mühendislik ve destek çalışmalarının ayrı ayrı seviyelendirildiğini anlattı. Yani bir kurumun proje yönetimi açısından düzeyi, mühendislik çalışmaları açısından düzeyi ayrı ayrı belirlenirdi.

Bugünkü CMMI'de ise, adındaki "Integration" kelimesine uygun olarak, gruplar arasında etkileşim vardır. Destek süreç alanları da iki grupta toplanır:

- **Temel destek süreç alanları:** Kalite sağlama ve konfigürasyon yönetimi.
- **İleri destek süreç alanları:** Kurumsal bütünleşik altyapı, neden-sonuç (nedensel) analizinin gerçekleştirilmesi ve karar analizlerinin yapılması.

11. People CMM

11.1 Tanım ve Amaç

Bu kısımda internet bağlantısı koptuğu için hoca konuyu kısa bir tekrarla yeniden başlattı. People CMM, insanlarla ilgili konuları tanımlayıp iyileştirmek isteyen, kurumdaki çalışanları da takip etmek isteyen kurumlar için geliştirilmiştir. Hocaya göre çalışanlar kurumda gerçekleştirilen işlerden ayrı görülemeyeceği için People CMM de CMMI'nin içine katılmıştır. People CMM, bir yazılım ve bilgi sistemleri organizasyonunun insan değerlerinin yönetimine ve geliştirilmesine yön verir.

People CMM'in hedefleri şunlardır:

- Yazılım geliştirme yeteneğini tanımlamak ve sağlamak.
- İş gücünün yeteneklerini artırarak kurumun yeteneklerini de artırmak.
- Sürekli bir iyileştirme içine girmek.
- Her bir çalışanın ve organizasyonun hedeflerini birlikte tutmak.
- İnsan değerlerini elde tutmak.

  Ders sonrası not: Hoca People CMM'in "geçen sene" CMMI'nin içine katıldığını söyledi. People CMM büyük olasılıkla SEI tarafından ayrı bir model olarak geliştirilmiştir; CMMI'nin 2018'de yayımlanan V2.0 sürümünde insanlarla ilgili uygulamalar büyük olasılıkla aynı çatı altında toplanmıştır. Bu birleşmenin kapsamı altyazıdan doğrulanamıyor.

11.2 People CMM Düzeyleri

Hoca, People CMM düzeylerinin de CMMI düzeyleriyle eşleştirilmiş olduğunu söyledi. Düzeyler şunlardır: başlangıç, tekrarlanabilir (repeatable), tanımlı (defined), yönetilen (managed) ve en iyileşen (optimizing). Hoca, dördüncü düzeye burada "sayısal yönetilebilir" denmediğine, yalnızca "yönetilen" denildiğine dikkat çekti.

Düzey 1: Başlangıç. Kaotik bir ortam vardır. Çalışanların hedefleriyle organizasyonun hedefleri arasında bir bağ kurulmaz. Çalışanlara yalnızca ne iş ürettikleri açısından bakılır. Performans yönetimi ve eğitim gibi konulara bakılmaz.

Düzey 2: Tekrarlanabilir. Çalışanların performanslarını tekrarlayabilmeleri için gerekli koşulları sağlayan kurumlar bu düzeydedir. Kurum şu konularla ilgilenir ve bunları sağlar:

- çalışanların tazminatı,
- eğitim,
- performans yönetimi,
- işe alım, personel sağlamadaki kurallar,
- çalışanlarla iletişim,
- çalışma ortamının ergonomik olması.

Düzey 3: Tanımlı. Bu düzeyde şu süreç alanları bulunur: katılımcı kültür, yeteneğe bağlı pratikler, kariyer geliştirme, yetenek geliştirme, iş gücü planlama, bilgi ve becerilerin analizi. Hoca bunları sorularla açtı:

- Katılımcı kültür: Kurumda "şirket kültürü" denilen bir yaklaşım var mı?
- Yeteneğe bağlı pratikler: Ekip elemanı seçiminde yetenekler göz önünde bulunduruluyor mu?
- Kariyer geliştirme: Çalışanlar için bir kariyer geliştirme planı var mı?
- Yetenek geliştirme ve eğitim: Eğitimler, çalışanların kariyer planlamasına ve kazandırılmak istenen yeteneklere göre planlanıyor mu?

Hoca ikinci ve üçüncü düzey arasındaki farkı eğitim üzerinden anlattı. "Kurumda şu işler yapılıyor, bununla ilgili eğitimler alınıyor" deniyorsa ikinci düzeydeyiz. Tanımlı düzey için ise eğitimler çalışanların kariyer planlamasına ve belirli yetenekleri kazandırma hedeflerine göre yapılmalıdır.

Düzey 4: Yönetilen. Kurumsal performans ayarlama, kurumsal yetenek yönetimi, takım temelli pratikler, ekip kurma ve danışmanlık (mentoring) varsa kurum bu düzeydedir. Hoca, CMMI'de de dördüncü düzeyde kurumsal yapıya bakıldığını hatırlattı. People CMM'de buna karşılık gelen şey, çalışanların ekip kurma becerisi ve ekip yapısının nasıl kurulacağına dair uygulamaların gerçekleştirilmesi ve ispatlanmasıdır. Üçüncü düzeyde kariyer ve yetenek geliştirme varken, dördüncü düzeyde kurumsal olarak bir yetenek yönetiminin olup olmadığına bakılır.

Hoca burada bir gözlemini paylaştı: Yazılım kurumları ekip yapısının önemini anladıkça, ekip çalışmasına yatkın, ortak proje geliştirme tarafı gelişmiş mezunlar istemektedir.

Düzey 5: En İyileşen. Sürekli iş gücü inovasyonu, koçluk (coaching) sistemi ve kişisel yetenek geliştirme bu düzeyde sağlanır. Sürekli iş gücü inovasyonundan kasıt yaratıcılıktır. Firmalarda projeler genellikle yukarıdan verilir. Ama yeni fikirlerin ortaya çıkabilmesi için çalışanların ortaya attığı projelere de imkân tanınır ve bunun için çeşitli aktiviteler düzenlenir. Koçluk sistemiyle belirli görevler için kişilere yeni yetenekler kazandırılır. Bu, kişisel yetenek geliştirmeyle yan yana gider; koçluk sistemi kişisel yetenek geliştirmeyi daha sağlanabilir kılar.

Düzey	Ad	Süreç alanları
1	Başlangıç	Kaotik ortam; yalnız üretilen işe bakılır
2	Tekrarlanabilir (Repeatable)	Tazminat, eğitim, performans yönetimi, işe alım, iletişim, ergonomik çalışma ortamı

Düzey	Ad	Süreç alanları
3	Tanımlı (Defined)	Katılımcı kültür, yeteneğe bağlı pratikler, kariyer geliştirme, yetenek geliştirme, iş gücü planlama, bilgi ve beceri analizi
4	Yönetilen (Managed)	Kurumsal performans ayarlama, kurumsal yetenek yönetimi, takım temelli pratikler, ekip kurma, danışmanlık
5	En İyileşen (Optimizing)	Sürekli iş gücü inovasyonu, koçluk, kişisel yetenek geliştirme

> **Ders sonrası not:** Hoca People CMM düzeylerini ilk sürümdeki adlarla verdi. People CMM'in 2. sürümünde büyük olasılıkla 2. düzeyin adı Managed, 4. düzeyin adı ise Predictable olarak değiştirilmiştir.

12. CMMI Bölümünün Toparlanması

Hoca bu bölümü şöyle bağladı: Kalite iyileştirme için ABD'de, Avrupa'da ve Uluslararası Standartlar Organizasyonu'nda (ISO) geliştirilmiş yazılım sertifika sistemleri vardır. Bunlardan ABD'de geliştirilen ve dünya çapında en çok kullanılan CMMI üzerinden süreç iyileştirme modeli anlatıldı. CMMI, Carnegie Mellon Üniversitesi'ndeki Software Engineering Institute tarafından takip edilen bir modeldir ve sürekli süreç iyileştirme döngüsünü esas alır. Beş düzeyin her birinde hedefin ne olduğu adım adım görüldü. En iyileştiren düzey aslında inovasyonla iç içedir; yeni fikirler ve geri beslemeler gerektirir.

Anahtar süreç alanları da düzeylerle eşleştirildi. Hocanın tanımıyla, CMMI'de anahtar süreç alanı denince, her bir düzeyi belirleyebilmek için o düzeyde hangi temel yazılım geliştirme işlemlerinin ve destek aktivitelerinin yapıldığını, proje yönetimi açısından hangi seviyede olunduğunu anlatan aktiviteler kastedilir.

13. Çevik Yaklaşımlar ve XP

13.1 Çevik Yaklaşımların Amacı

İkinci aradan sonra hoca, ilk derslerde adını andığı test güdümlü geliştirmeye (test driven development) ve çevik süreçlerde kalite yönetiminin nasıl yapıldığına kısaca değineceğini söyledi. Çevik yaklaşımlar yazılım mühendisliği dersinde yazılım geliştirme süreç modelleri arasında görülmüştü.

Soru (hocadan): Çevik yaklaşımla yazılım geliştirme süreç modellerinden isim söyler misiniz? **Cevap:** Öğrenciler Scrum, XP ve Kanban'ı saydı. Hoca bunların yazılım mühendisliği dersinde birbirinden farklarıyla görüldüğünü hatırlattı.

Çevik yaklaşımlar, artan müşteri taleplerini daha hızlı karşılamak için geleneksel sistem geliştirme modellerine alternatif olarak ortaya çıkmıştır. Gereksinim değişikliklerine daha hızlı uyum sağlarlar. En önemli özelliklerinden biri de müşteriyle ilişkinin hızlı olması, yani müşterinin yazılım ekibinin içinde yer almasıdır.

13.2 XP'nin Dört Temel Değeri ve Pratikleri

Hocaya göre çevik yaklaşımda ilk kullanılan yazılım geliştirme süreç modeli XP'dir (Extreme Programming). XP dört temel değer çerçevesinde oluşturulmuştur: basitlik, iletişim, geri besleme ve cesaret. Hoca XP pratiklerini bu değerlere bağlayarak anlattı:

- **Planlama oyunu.**
- **Küçük ve kısa aralıklı sürümler (release):** Geri beslemeyi artırır. İş, biten artımlarla devam eder.

- **Sistem metaforu.**
- **Basit tasarım:** Tasarım basit kurulur ve küçük sürümlerle geliştirilir.
- **Test odaklılık.**
- **Devamlı yeniden tasarım (refactoring):** Hocaya göre basitlik, geri besleme ve cesareti birlikte anlatan bir kavramdır; geri beslemeyi gerektirir.
- **Eşli programlama (pair programming):** İletişimin fazla olması ve kodun birlikte oluşturulması.
- **Ortak kod mülkiyeti.**
- **Sürekli tümleştirme (continuous integration).**
- **Devam ettirilebilir hız:** Haftada 40 saat gibi bir çalışma süresi.
- **Müşterinin ekip üyesi olması:** Böylece geri besleme daha hızlı olur.
- **Kodlama standartları.**

Hoca değerlerle pratikler arasındaki bağı şöyle kurdu: Basitlik basit tasarım ve sistem metaforuyla, iletişim müşterinin ekipte olması ve eşli programlamayla, geri besleme küçük adımlar ve kısa aralıklı sürümlerle, cesaret ise yeni yaklaşımları küçük eklemelerle hızlıca hayata geçirebilmekle ilgilidir. Bu pratiklerin birçoğu genel olarak çevik modellerin de özelliğidir.

Devam ettirilebilir hız, People CMM kurallarıyla örtüşür. Fazla mesai kavramı yoktur. Çalışanlar haftada 40 saat, normal bir eforla çalışır; böylece süreklilik sağlanır.

13.3 Scrum ile XP Arasındaki Fark

Hoca iki model arasındaki farkı şöyle anlattı: Scrum, XP'den sonra geliştirilmiştir. Scrum'da koşu (sprint) kavramı vardır ve bir koşu için iki hafta gibi bir süre verilir. Her koşuda tanımlanan işler eklenerek ilerlenir. XP'de ise koşulardan söz edilmez. Ürün küçük artımlarla, yeni sürümlerle geliştirilir ve iyileştirilir; iş sürekli tümleştirme şeklinde ilerler. Yani XP artımlı (incremental) bir modeldir.

> **Ders sonrası not:** Scrum'da koşu süresi büyük olasılıkla en fazla bir aydır; iki hafta yaygın bir seçimdir. XP'de de büyük olasılıkla bir-iki haftalık iterasyonlar tanımlanır. Dolayısıyla iki model arasındaki fark "koşu var ya da yok" değil, işin planlanma ve teslim biçimindedir.

14. Çevik Modellerde Kalite: Geleneksel Modellerden Farklar

Hoca, çevik modellerin kalite açısından geleneksel yazılım geliştirme modellerinden farklarını sıraladı:

- **Dinamik kalite değerlendirmesi statik değerlendirmeden daha fazla uygulanır.** Yani kod incelemeden çok test yapılır.
- **Kalite işlemleri sürecin daha erken adımlarında başlar.** Klasik modellerde kalite işlemleri, özellikle test, yazılım geliştirmenin sonuna doğru yapılır. Çevik modellerde ise ilk adımlardan itibaren kalite göz önünde bulundurulur; test bile en başta planlanmaya başlar.
- **Kalite aktiviteleri daha kısa aralıklarla, daha sık yapılır.** Her küçük parça geliştirildiğinde test yapılır ve kaliteyle ilgili gözden geçirme daha sık yapılır. Müşteri de zaten ekibin içindedir.
- **Kullanıcı hikâyeleri (user story), birim testler ve müşteri geri beslemeleriyle doğruluk ve süreklilik sağlanır.**

Bu yüzden hocaya göre çevik modellerde kalite açısından geleneksel modellere göre daha fazla işlem yapılabilir.

15. Statik ve Dinamik Doğrulama

"Dinamik değerlendirme statikten daha fazla" maddesinde hoca öğrencilerin eski bilgilerini yokladı.

Soru (hocadan): Statik doğrulama (static verification) nedir? Kalite aktivitelerimizden hangisi statik doğrulamaya örnek verilebilir?

Öğrencilerden gelen ilk cevaplar şöyleydi: Bir öğrenci doğrulama için yapılan bir şey olduğunu hatırladı ama tam çıkaramadı. Bir başka öğrenci "programı çalıştırmadan kod üzerinden analiz yapmak" dedi. Hoca tanımın doğru olduğunu, ama aktivitenin adını istediğini söyledi. "Kaynak kod üzerinden analiz yapıyoruz" ve "hata ayıklama (debugging) mı?" cevapları geldi. Hoca hata ayıklamanın doğru olmadığını belirtti.

Cevap (hoca): İnceleme (inspection) yapıyoruz; örneğin kod incelemesi (code inspection). Test dinamiktir; inceleme ise statiktir. Statik doğrulamada inceleme yapılır.

Ders sonunda bir öğrenci aynı konuya geri döndü.

Soru: Statik doğrulamada kodu analiz etmek ile kodu incelemek (inspection) arasında ne fark var, aynı şey olmaz mı?

Cevap: İnceleme daha ayrıntılı bir değerlendirmedir. Derste kalite aktiviteleri arasında gözden geçirme (review) ile inceleme arasındaki farklar anlatılmıştı. "Analiz yapmak" denince gözden geçirme de anlaşılabilir. Statik doğrulamada derste verilen örnek inceleme olduğu için hoca o adı duymak, öğrencilerin notlara bakıp bakmadığını görmek istemişti. Analiz de aynı anlama gelebilir, ama derste geçen ad kod incelemesidir (code inspection).

16. Test Güdümlü Yazılım Geliştirme (TDD)

16.1 Tanım

Test güdümlü yazılım geliştirme (Test Driven Development, TDD), az önce sayılan çevik kalite farklarını sağlayacak şekilde çalışan bir yaklaşımdır. Hoca bunu nesneye yönelik yazılım geliştirme üzerinden anlattı: Gereksinim analizi sırasında bir kullanıcı senaryosu (use case) oluşturulur. Hemen ardından bu senaryonun test senaryosunun nasıl olması gerektiği düşünülür. Testler küçük ve hızlı iterasyonlarla, yazılım geliştirme araçları (tool) kullanılarak hazırlanır. Her bir kullanıcı senaryosu yazıldığında onun test senaryosu da yazılarak ilerlenir. TDD'ye test yönelimli ya da test güdümlü yazılım geliştirme denir ve çevik uygulamalarda geçerli bir yöntemdir.

16.2 TDD Adımları

Hoca TDD'nin adımlarını şöyle verdi:

- **Adım 1:** Birim test senaryoları (unit test case) yazılır. Her bir küçük modülü test edecek senaryo belirlenir.
- **Adım 2:** Yazılmış test senaryoları, henüz kod yazılmadığı için başarısız olduklarını görmek üzere çalıştırılır.
- **Adım 3:** Kod yazılarak test senaryolarının geçmesi sağlanır.
- **Adım 4:** Test senaryoları tekrar koşturularak yazılımın testten geçtiği görülür. Geçemiyorsa kod yeniden düzenlenir.

Hoca özünü şöyle özetledi: Önce bir test senaryosu oluşturulur ve yazılımı geliştirmeden önce bu senaryonun nasıl çalıştırılacağı planlanır. Kod yazılır yazılmaz test senaryosu koda uygulanır.

> **Ek bilgi (derste anlatılmadı):** TDD döngüsü literatürde büyük olasılıkla "kırmızı-yeşil-yeniden düzenle" (red-green-refactor) diye anılır: önce başarısız bir test (kırmızı), sonra testi geçiren en basit kod (yeşil), ardından testler geçerken kodun yapısının iyileştirilmesi (refactor). Bu tanımda yeniden düzenleme, test geçtikten sonra davranışı değiştirmeden yapılır; test geçmiyorsa yapılan iş hata düzeltmedir.

16.3 Şirketlere Yansıması

Hoca TDD'nin şirket yapısına etkisini anlattı. Çevik yaklaşımlar yaygınlaştıkça şirketlerde test grubunun ayrı bir grup olmadığı, sistem analizi ekibinin içinde yer aldığı görülmektedir. Çünkü test senaryoları sistem analizi yapılırken geliştirilir. Bu yüzden gereksinim analizini yapan sistem analizi ekibi ile test senaryolarını geliştiren, kullanıcı testini yapan ekip aynı yerde, iç içe çalışacak şekilde görevlendirilir.

17. TDD Üzerine Bir Araştırma

Hoca TDD konusunda yapılan ilk çalışmalardan birini aktardı. Çalışmanın sahibi Hakan Erdoğmuş'tur. Erdoğmuş IEEE Software dergisinin editörlüğünü yapmış, Boğaziçi Üniversitesi mezunu bir yazılımcıdır; hoca onu yazılım mühendisliği konferanslarına davetli konuşmacı olarak çağırdıklarını da söyledi.

Erdoğmuş, 24 lisans öğrencisiyle "önce test" ve "sonra test" yaklaşımlarını karşılaştıran bir araştırma yapmıştır. Hocanın aktardığı bulgular şunlardır:

- TDD programcı üretkenliğini artırmıştır; programcılar daha hızlı çalışmıştır.
- Kalite konusunda belirgin bir fark görülmemiş, TDD daha iyi kalite sağlayamamıştır.
- Önce test yaklaşımı, programcıyı kodları ve test sonuçlarını yedeklemeye iten bir güç olmuştur.

  Ders sonrası not: Hoca kalite bulgusunu bir yerde "en azından bir farklılık yok", başka bir yerde "kalite açısından biraz daha geride" diye aktardı. Çalışmanın sonucu büyük olasılıkla kalitede anlamlı bir fark bulunmadığı yönündedir. Ayrıca hoca bu çalışmaların "yaklaşık 11 sene öncesinden beri" sürdüğünü söyledi; Erdoğmuş ve arkadaşlarının çalışması büyük olasılıkla 2005'te yayımlanmıştır, yani ders tarihinden yaklaşık 15 yıl öncedir.

Örnek Sorular

Soru 1 (derste sözlü, 00:09:06): Yazılım standartlarını hangi türlere göre sınıflandırmıştık?

> **Çözüm (derste verildi):** İki türlü: kaynağına göre (standardı oluşturan ve destekleyen kurum: IEEE, ISO, DoD) ve yazılım geliştirme sürecine göre (desteklediği süreç adımı: geliştirme planı, gereksinim, tasarım). Örneğin IEEE 830'un kaynağı IEEE'dir; süreç açısından yazılım gereksinim analizini kontrol altında tutar.

Soru 2 (derste sözlü, 01:41:31): Çevik yaklaşımla yazılım geliştirme süreç modellerinden isim söyleyiniz.

> **Çözüm (derste verildi):** Scrum, XP ve Kanban.

Soru 3 (derste sözlü, 01:52:47): Statik doğrulama (static verification) nedir? Kalite aktivitelerimizden hangisi statik doğrulamaya örnek verilebilir?

> **Çözüm (derste verildi):** Statik doğrulama, programı çalıştırmadan kod üzerinden yapılan değerlendirmedir. Örnek aktivite inceleme (inspection), örneğin kod incelemesidir (code inspection). Test ise programı çalıştırarak yapıldığı için dinamik doğrulamadır. Hata ayıklama (debugging) doğru cevap kabul edilmedi.

Soru 4 (derste sözlü, 02:05:37): Statik doğrulamada kodu analiz etmek ile kod incelemesi (inspection) arasında fark var mı?

> **Çözüm (derste verildi):** İnceleme daha ayrıntılı bir değerlendirmedir. "Analiz" denince gözden geçirme (review) de anlaşılabilir. Derste statik doğrulama örneği olarak inceleme verildiği için beklenen ad kod incelemesidir.

Dersin Toparlaması

Bu derste kalite güvencesinin son halkası olan süreç iyileştirme ve sertifikasyon ele alındı. Standartlar, süreç modeli, gözden geçirme, inceleme, test, konfigürasyon yönetimi ve ölçüm arşivi bir yazılım evinin kaliteyi sağlamak için yaptığı işlerdir. Sertifika ise bunların ölçülebilir biçimde kanıtlanmasıdır. CMMI, ürünün kalitesini üreten sürecin belirlediği varsayımıyla beş olgunluk düzeyi tanımlar: kaotik başlangıçtan belgelenmiş proje yönetimine, kurumsal standart süreç ve risk yönetimine, ölçüm arşivine dayalı sayısal yönetime ve yenilikçi sürekli iyileştirmeye. Her düzeyin anahtar süreç alanları o düzeyde yapılması gereken işleri söyler. People CMM aynı mantığı çalışanlara uygular: tazminat ve eğitimden kariyer planlamasına, ekip kurmaya ve koçluğa.

Dersin ikinci kısmında çevik modellerde kalitenin geleneksel modellerden farkı görüldü: Test daha erken, daha sık ve statik incelemeden daha ağırlıklı yapılır. Test güdümlü geliştirme bu

farkları uygulamaya döker; önce birim test yazılır, başarısız olduğu görülür, sonra testi geçiren kod yazılır. Hoca son olarak bu yaklaşımın şirketlerde test ekibini sistem analizi ekibinin içine taşıdığını ve yapılan bir araştırmada üretkenliği artırdığını, kalitede ise belirgin fark yaratmadığını aktardı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.